

ARM register conventions

Name	Register No	Usage	Preserved on call?
a1-a2	0-1	Argument/return result/scratch register	no
a3-a4	2-3	Argument/scratch register	no
v1-v8	4-11	Variables for local routine	yes
ip	12	Intra-procedure- call scratch register	no
sp	13	Stack pointer	yes
lr	14	Link register	yes
pc	15	Program counter	n.a

MIPS register Convention

Name	Register No	Usage	Preserved on call?
\$zero	0	The constant value 0	n.a
\$v0-\$v1	2-3	Values for results and expression evaluation	no
\$a0-\$a3	4-7	Arguments	no
\$t0-\$t7	8-15	Temporaries	no
\$s0-\$s7	16-23	Saved	yes
\$t8-\$t9	24-25	More temporaries	no
\$gp	28	Global pointer	yes
\$sp	29	Stack pointer	yes
\$fp	30	Frame pointer	yes
\$ra	31	Return address	yes

32-bit immediate operand

- Although constants are frequently short and fit into 16 bit field, sometimes they are bigger. The MIPS instruction set includes the instruction ***load upper immediate*** (*lui*)
- Specifically to set the upper 16bits of a constant in a register, allowing a subsequent instruction to specify the lower 16bits of the constant.

m/c code of lui \$t0,255 ;t0 is register 8

001111	00000	01000	0000 0000 1111 1111
--------	-------	-------	---------------------

Contents of register \$t0 after executing lui \$t0,255

0000 0000 1111 1111	0000 0000 0000 0000
---------------------	---------------------

- Loading 32-bit constant

- What is the MIPS assembly code to load 32-bit constant into register \$s0?

0000 0000 0011 1101 0000 1001 0000 0000

First load the upper 16 bits which is 61 in decimal ,using lui;

Lui \$s0,61

The value of register \$s0 afterwards is

0000 0000 0011 1100 0000 0000 0000 0000

Add the lower 16 bits whose decimal is 2304

ori \$s0,\$s0,2304

The final value in the register \$s0

0000 0000 0011 1101 0000 1001 0000 0000

Compiling a String Copy Procedure

```
void strcpy(char x[],char y[])
{ int i;
  i=0;
  while((x[i]=y[i])!='\0')
    i+=1;
}
```

- In ARM:

Assume the base addresses for arrays x and y are found in r0 and r1, while i is in r4. strcpy adjusts the stack pointer and then saves the saved register r4 on the stack

strcpy:

SUB sp,sp,#4 ;adjust stack for 1 more item

STR r4,[sp,#0] ;save r4

To initialize i to 0, the next instruction sets r4 to 0 by adding 0 to 0 and placing that sum in r4:

MOV r4,#0 ;i=0+0

L1:ADD r2,r4,r1 ;address of y[i] in r2 {this is the beginning of the loop,the address of y[i] is first formed by adding i to y[](assume array of bytes).

To load the character in y[i]

LDRB r3,[r2,#0] ;r3=y[i] and set condition flags {load register byte;loads a byte from memory

ADD r12,r4,r0 ;address of x[i] in r12

STRB r3,[r12,#0] ;x[i]=y[i]

BEQ L2 ;if y[i]==0; go to L2

Increment i and loop back

ADD r4,r4,#1 ;i=i+1

B L1 ;go to L1

If we don't loop back it was the last character of the strings ;we restore r4 and the stack pointer,and then return

L2:LDR r4,[sp,#0] ;y[i]==0;end of string ,restore old r4

ADD sp,sp,#4 ;pop 1 word off stack

MOV pc,lr ;return

In MIPS:

strcpy:

addi \$sp,\$sp,-4

sw \$sp,0(\$sp)

add \$s0,\$zero,\$zero,

L1:ADD \$t1,\$s0,\$a1 ;address of y[i] in \$t1

lb \$t2,0(\$t1) ;\$t2=y[i]

add \$t3,\$s0,\$a0 ;address of x[i] in \$t3

sb \$t2,0(\$t3) ;x[i]=y[i]

beq \$t2,\$zero ,L2

addi \$s0,\$s0,1

J L1

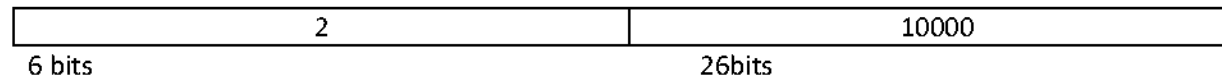
L2: lw \$s0,0(\$sp)

addi \$sp,\$sp,4

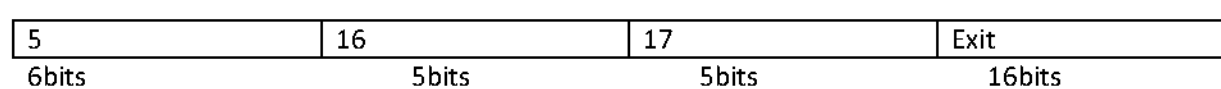
jr \$ra

Addressing in Branches and Jumps

J 10000 ;go to location 10000



bne \$s0,\$s1,Exit ;go to Exit if \$s0 ≠ \$s1



If the address of the program is bigger than 16 bits than $PC = Reg + \text{branch address}$, sum becomes 32 bits

Branching far away: Given `beq $s0,$s1,L1`, replace with a pair of instruction that offer a much greater branching distance.

Short address conditional branch:

`bne $1,$s0,L2`

`J L1`

`L2:`

MIPS Addressing Modes:

- 1) Register addressing (where the operand is a register)
- 2) Base or Displacement addressing (where the operand is at the memory location whose address is the sum of the register and a constant in the instruction)
- 3) Immediate addressing (operand is the constant within the instruction itself)
- 4) PC relative addressing (where the address is the sum of the PC and a constant in the instruction)
- 5) Pseudodirect addressing (where the jump address is the 26 bits of the instruction concatenated with the upper bits of the PC)

- 1)convert into assembly for m/c instruction 00af8020(hex)

0000 0000 1010 1111 1000 1000 0010 0000

Find the op field

31-29 and 28-26 are 000 ,000 ,hence R-format instruction

000000 00101 01111 10000 00000 100000

op rs rt rd shamt funct

Bit 5-3 ,100 and bits 2-0,000,hence it represent add

The decimal values are 5 for rs and 15 for rt,16 for rd(shamt is unused).these numbers represents registers \$a1,\$t7 and \$s0

add \$s0,\$a1,\$t7

Translating and starting a program

C-program----→COMPILER----→Assembly program--→ASSEMBLER--
→object:m/c language module---→LINKER--→exe m/c language program--
→LOADER--→memory



object:library routine(m/c language)

Source file-x.c

Assembly file-x.s

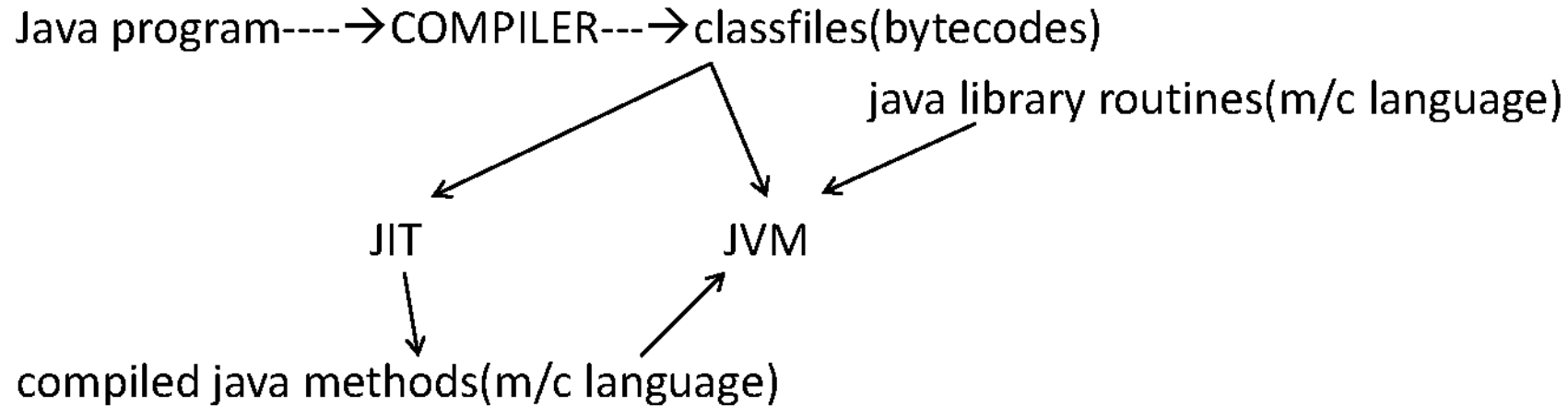
Object file-x.o

Statically linked library routines are x.a and dynamically linked library routes are x.so

Executable files by default are called a.out.

MS-DOS uses .C,.ASM,.OBJ,.LIB,.DLL,and .EXE

Compiling java program



JVM-s/w interpreter, can execute bytecodes, it's a program that simulates an ISA. portable and found in devices –mobile phones to internet browsers.

To preserve portability and improve execution speed the next phase is JIT

JIT-compiler that operates on runtime, translating the interpreted code segments into the native code of the computer.

Arithmetic for computers

- Two's complement representation: The positive and negative numbers 32 bit numbers can be represented as
- $(x_{31} \times -2^{31}) + (x_{30} \times 2^{30}) + (x_{29} \times 2^{29}) + \dots + (x_1 \times 2^1) + (x_0 \times 2^0)$
- The sign bit is multiplied by -2^{31} and the rest of the bits are then multiplied by positive versions of their respective base values.
- Decimal value of two's complement number:

1111 1111 1111 1111 1111 1111 1111 1100

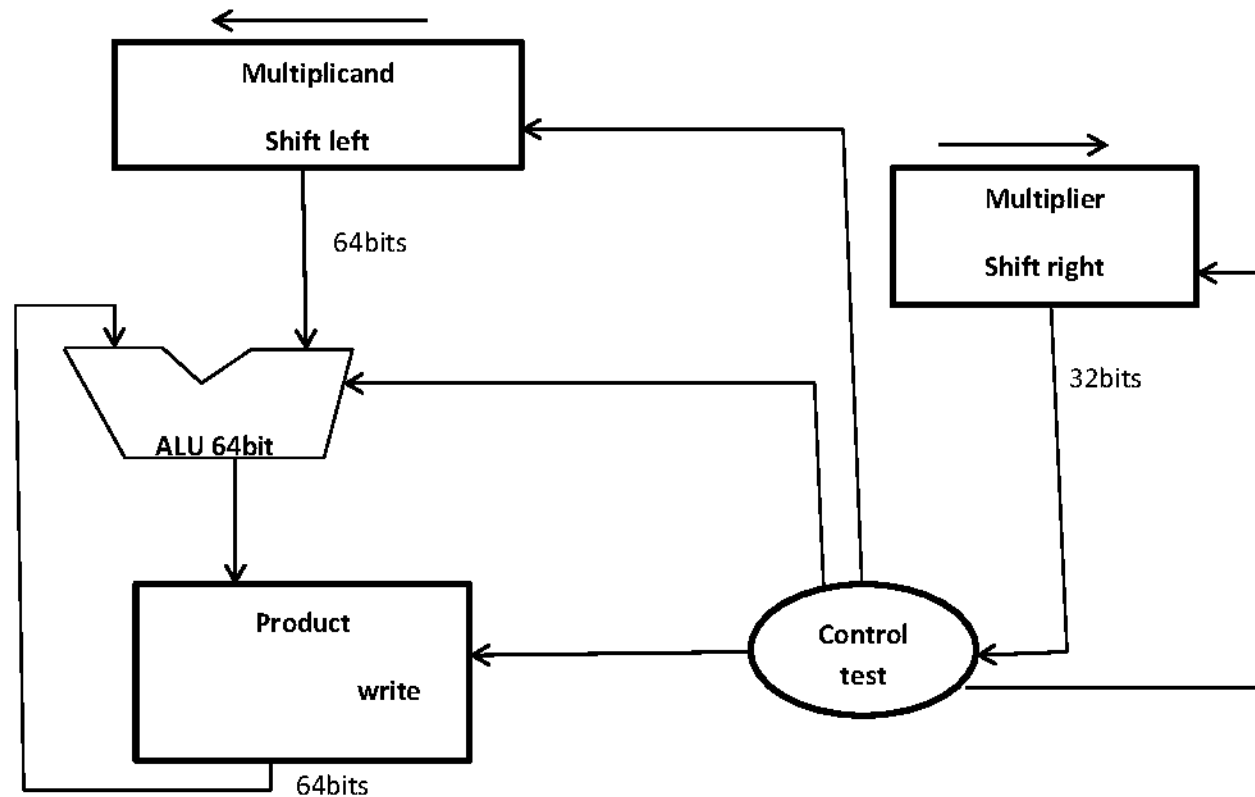
Multiplication: Sequential version of the Multiplication Algorithm and Hardware

Let's assume that the multiplier is in the 32-bit Multiplier register and that the 64-bit Product register is initialized to 0.

We need to move the multiplicand left one digit each step, as it may be added to the intermediate products.

Over 32 steps, a 32 bit multiplicand would move 32 bits to the left. Hence, we need a 64bit multiplicand register, initialized with the 32 bit multiplicand in the right half and zero in the left half. This register is then shifted left 1 bit each step to align the multiplicand with the sum being accumulated in the 64-bit Product register.

First version of the multiplication H/w



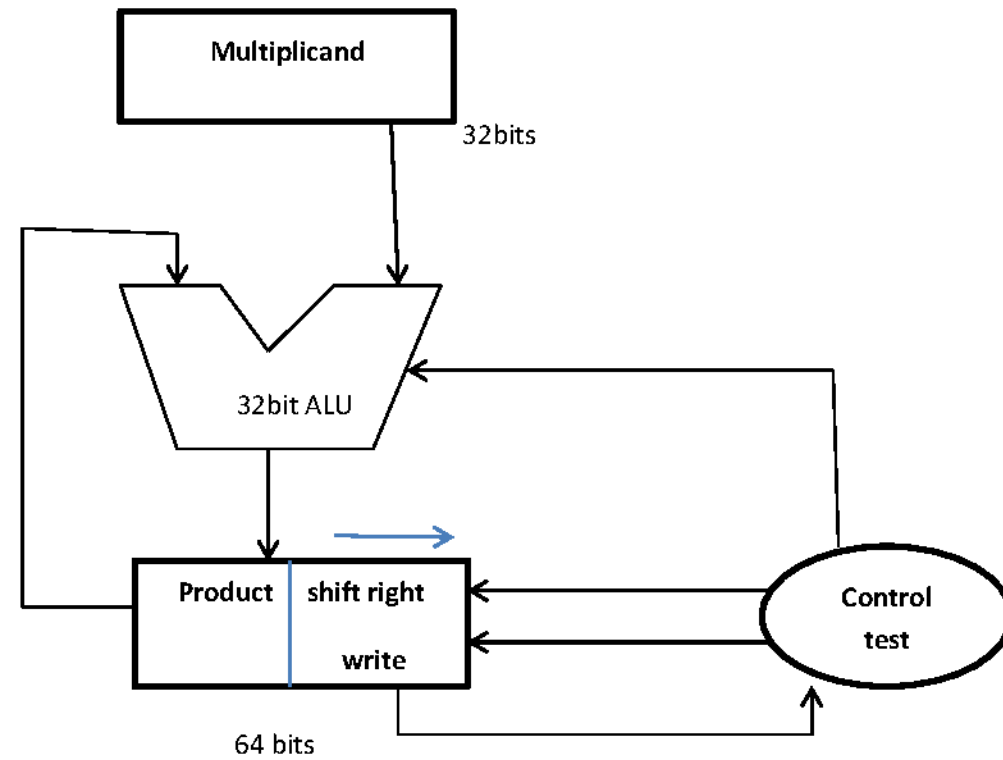
First multiplication algorithm

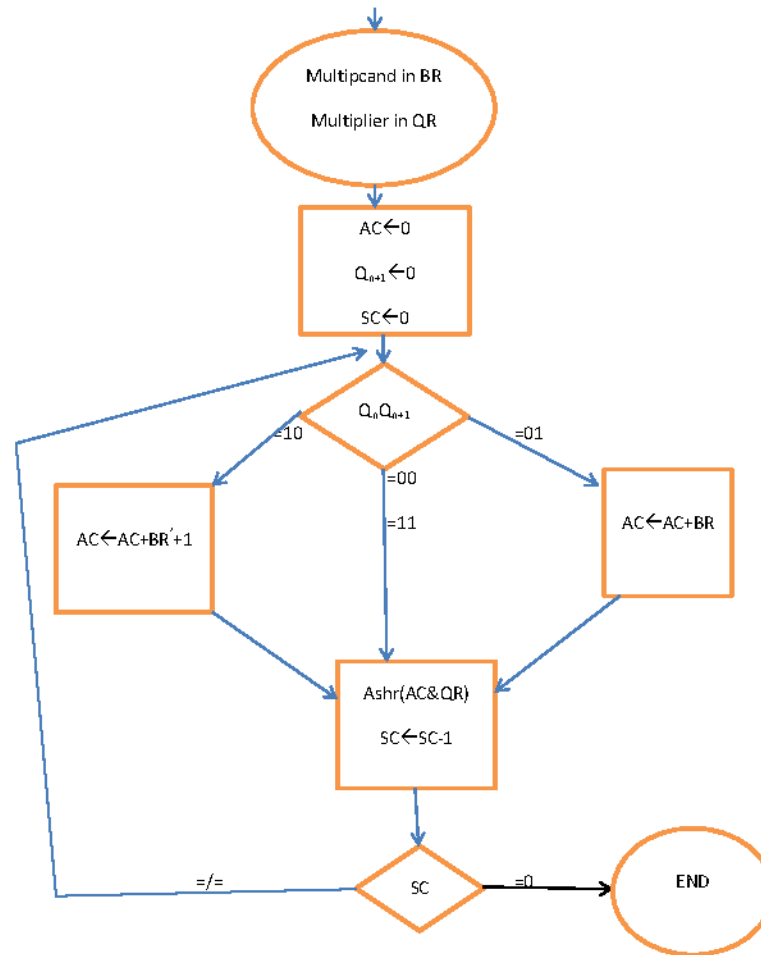
- [flowchart multiply.pdf](#)
- Three basic steps needed for each bit. These three basic steps are repeated 32 times to obtain the product. If each step took a clock cycle, this algo would require almost 100 clock cycles to multiply two 32 bit numbers.

Multiply 0010 X 0011

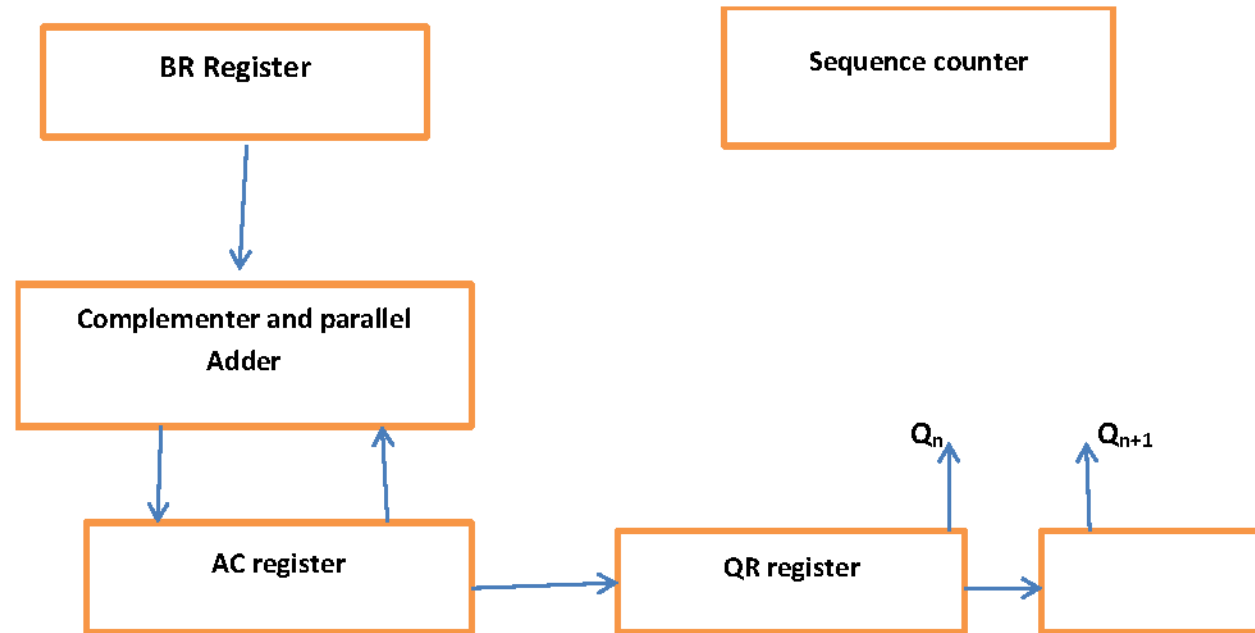
Iteration	Step	Multiplier	Multiplicand	Product
0	Initial value	0011	0000 0010	0000 0000
1	1a:1=>Prod=Prod + Multiplicand	0011	0000 0010	0000 0010
	2:shift left multiplicand	0011	0000 0100	0000 0010
	3:shift right multiplier	0001	0000 0100	0000 0010
2	1a:1=>Prod=Prod + Multiplicand	0001	0000 0100	0000 0110
	2:shift left multiplicand	0001	0000 1000	0000 0110
	3:shift right multiplier	0000	0000 1000	0000 0110
3	1a:1=>No operation	0000	0000 1000	0000 0110
	2:shift left multiplicand	0000	0001 0000	0000 0110
	3:shift right multiplier	0000	0001 0000	0000 0110
4	1a:1=>No operation	0000	0001 0000	0000 0110
	2:shift left multiplicand	0000	0010 0000	0000 0110
	3:shift right multiplier	0000	0010 0000	0000 0110

Refined version of Multiplication H/w

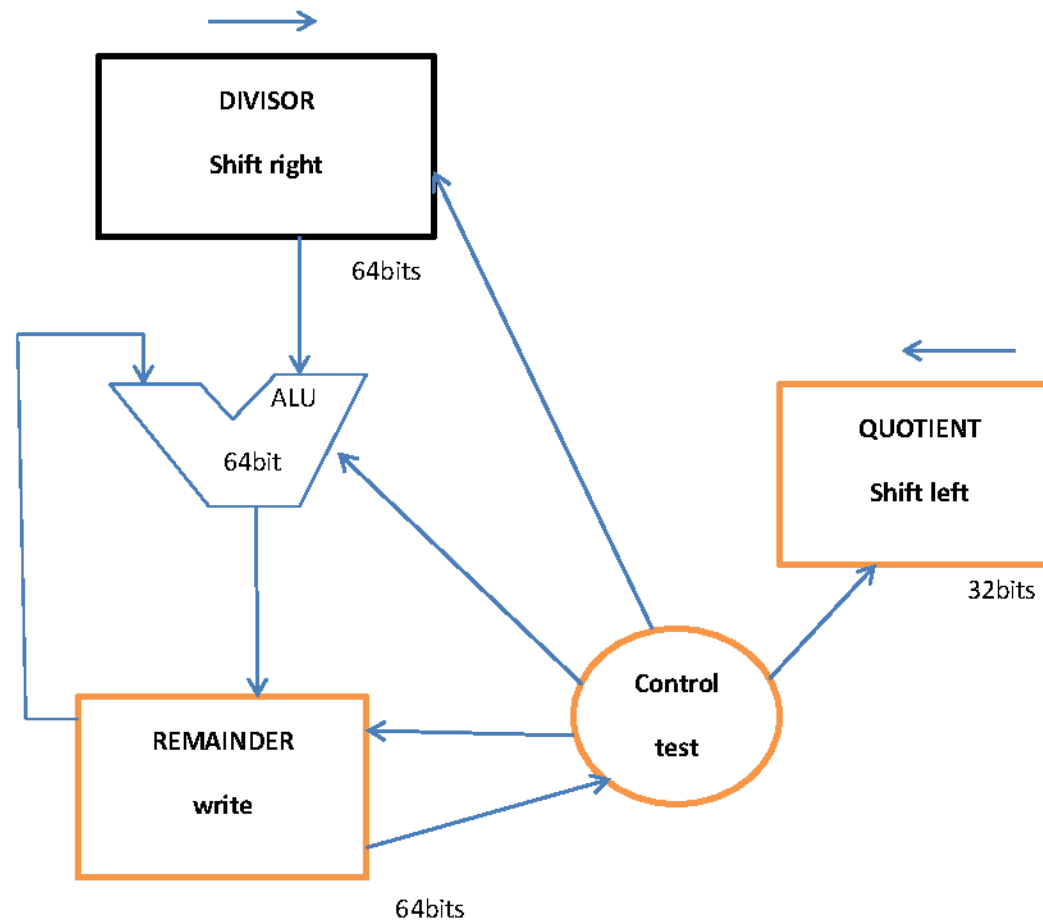




h/w for Booth Algo



First version of division h/w



Floating Point

Reals in mathematics:

3.14159265....(π)

2.71828...(e)

0.000000001 or 1.0×10^{-9}

3,155,760,000 or 3.15576×10^9

The last number did not represent a small fraction, but it was bigger than we could represent with a 32 bit signed integer.

The alternative notation of the last two numbers is called *scientific notation*.

Which has a single digit to the left of the decimal point.

A number in scientific notation that has no leading 0s is called a normalized number, which is the usual way to write it. 1.0×10^{-9} is a normalized scientific notation but 0.1×10^9 and 10.0×10^{10} are not

Note: all numbers in decimal

- We can also show binary numbers in scientific notation. 1.0×2^{-1}

Floating point: computer arithmetic that represent numbers in which the binary point is not fixed

In scientific notation $(1. xxxxxx)_2 \times 2^{yyyy}$

Advantages of scientific notation in normalized form:

- 1)It simplifies exchange of data that includes floating point numbers.
- 2)It simplifies the floating point arithmetic algorithms.
- 3)Increases the accuracy of the numbers that can be stored in a word.

31	30-23	22-0
S(1 bit)	Exponent(8 bits)	Fraction(23 bits)

General Form:

$$(-1) \times F \times 2^E$$

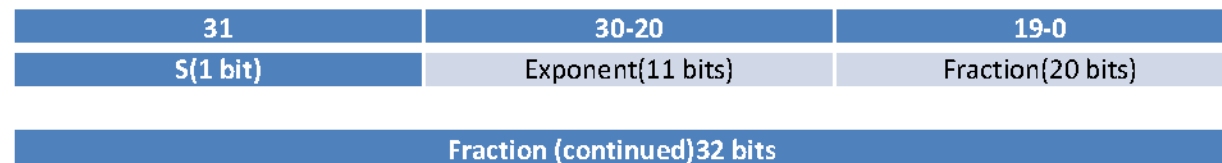
- These chosen sizes of exponent and fraction give MIPS computer arithmetic an extra ordinary range. Fractions almost as small as 2.0×10^{-38} and as large as 2.0×10^{38}

Overflow: positive exponent becomes too large to fit in the exponent field.

Underflow: negative exponent becomes too large to fit in the exponent field.

Double precision: Floating point value represented two 32 bit words.

Single precision: Floating point value represented in a single 32 bit word.



MIPS double precision:large number: 2×10^{308}

Small number: 2×10^{-308}

These formats go beyond MIPS:they are part of IEEE754 floating point standard found in virtually every computer invented since 1980.

To pack into more bits into significand(fraction),the IEEE754 makes the leading 1 bit of normalized binary numbers implicit.

Hence,the number is actually 24 bits long in single precision(1 implied+23 bit fraction).

For double precision for 53 bits long (1+52).

General form:

$$(-1)^S \times (1 + \textit{fraction}) \times 2^E$$

Where the bits of the fraction represents a number between 0 and 1 and E specifies the value in the exponent field.

If we number the bits of the fraction from left to right s1,s2,s3...then the value is

$$(-1)^S \times (1 + (s1 \times 2^{-1}) + (s2 \times 2^{-2}) + \dots) \times 2^E$$

- 1.0×2^{-1}

31	30-23	22-0
0	11111111	0000000000.....

- $1.0 \times 2^{+1}$

31	30-23	22-0
0	0000001	0000000000.....